

CPSC 250

Class Constructors

Edward Brash

1 Review

In the previous class examples, the constructor was kept as simple as possible.

For example:

```
def __init__(self):
    self.name = "none"
    self.rating = -1
    self.price = "none"
    self.cuisine = "none"
```

This constructor creates an object and assigns default values to all member variables. That works, but it is not always convenient.

2 The Problem

Often, when we create an object, we already know some or all of the information that should go inside it.

For example, rather than writing:

```
moes = Restaurant()

moes.set_name("Moe's")
moes.set_rating(3)
moes.set_price("***")
moes.set_cuisine("Mexican")
```

it would be nicer to write:

```
moes = Restaurant("Moe's", 3, "***", "Mexican")
```

This is possible by adding parameters to the constructor.

3 Constructor Parameters

We can define the constructor as:

```
class Restaurant:

    def __init__(self, name, rating, price, cuisine):
        self.name = name
        self.rating = rating
        self.price = price
        self.cuisine = cuisine
```

Now the object can be initialized directly:

```
moes = Restaurant("Moe's", 3, "***", "Mexican")
```

4 Important Notes

4.1 Parameter Names

It is common to use the same names for constructor parameters and instance attributes:

```
def __init__(self, name, rating, price, cuisine):
    self.name = name
    self.rating = rating
    self.price = price
    self.cuisine = cuisine
```

This is not a conflict.

The name on the right side is the parameter. The name on the left side after `self.` is the instance attribute.

4.2 Documentation

It is good practice to include documentation explaining the parameters.

```
class Restaurant:
    # Represents a restaurant.
    #
    # Parameters:
    #     name: Name of the restaurant.
    #     rating: Rating from 1 to 10.
    #     price: Price category such as "$", "$$", or "$$$".
    #     cuisine: Type of cuisine.
```

This documentation can be viewed using:

```
help(Restaurant)
```

5 Default Constructor Values

Sometimes we want to provide default values for some constructor arguments.

Example:

```
class Restaurant:

    def __init__(self, name, cuisine, rating=-1, price="none"):
        self.name = name
        self.cuisine = cuisine
        self.rating = rating
        self.price = price
```

Now we can write:

```
moes = Restaurant("Moe's", "Mexican")
```

and the object will use default values for `rating` and `price`.

6 Important Rule About Defaults

Non-default parameters must come before default parameters.

This is valid:

```
def __init__(self, name, cuisine, rating=-1, price="none"):
```

This is not valid:

```
def __init__(self, name="none", cuisine, rating=-1):
```

Python needs all required parameters first, followed by optional parameters.

7 Overriding Default Values

Default values are convenient, but they can still be overridden.

```
panera = Restaurant("Panera", "Soups and Sandwiches", 4, "$$$")
five_guys = Restaurant("Five Guys", "Burgers", 10, "$$$$$")
```

These calls supply explicit values for all fields.

8 Best Practice: Give Defaults to All Member Variables

A common design choice is to make sure every member variable has a reasonable value after construction.

This prevents partially initialized objects.

For example:

```
class Restaurant:

    def __init__(self, name="none", cuisine="none",
                  rating=-1, price="none"):
        self.name = name
        self.cuisine = cuisine
        self.rating = rating
        self.price = price
```

Then all of the following make sense:

```
Restaurant()
Restaurant("Moe's")
Restaurant("Moe's", "Fake Mex")
Restaurant("Moe's", "Fake Mex", 6)
Restaurant("Moe's", "Fake Mex", 6, "$$")
```

9 Internal Helper Methods

Another best practice is to use an underscore to indicate methods that are intended to be internal.

For example, suppose we define a rectangle:

```
class Rectangle:

    def __init__(self, length=1, width=1):
```

```
self.length = length
self.width = width

def _get_area(self):
    return self.length * self.width

def resize(self, new_length):
    area = self._get_area()
    self.length = new_length
    self.width = area / new_length

def print_rectangle(self):
    print(f"Length: {self.length:.3f}")
    print(f"Width: {self.width:.3f}")
    print(f"Area: {self._get_area():.3f}")
```

The method `_get_area()` is still technically accessible from outside the class, but the underscore indicates:

This method is intended for internal use.

10 A Note About Naming Conventions

When we see a method beginning with an underscore, such as:

```
_get_area()
```

we should not assume it is magical or special.

It is mostly a convention.

Python programmers use this convention to communicate design intent.

11 Key Takeaways

- Constructors initialize objects.
- Constructors can take parameters.
- Parameter values can be assigned to instance attributes.
- Default parameter values make object creation more flexible.
- Required parameters must come before default parameters.
- Documentation helps users understand constructor parameters.
- Leading underscores indicate internal-use methods by convention.